



東華大學
DONGHUA UNIVERSITY

大数据技术

DATA

计算机科学与技术学院

主讲教师：宋晖

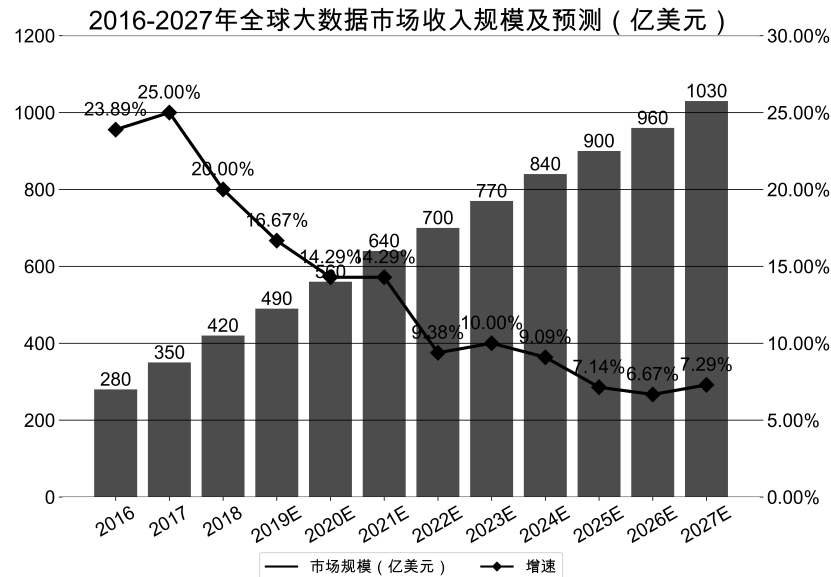
- Volume（规模性）
 - 数据的存储与计算需要耗费海量规模的资源
 - 卫星收集的数据达到32PB、新浪微博日活跃人数达到1.65 亿人
- Velocity（高速性）
 - 增长速度快，需要及时处理
 - 支付宝“双11”夜，0点支付峰值：25.6 万笔/秒，上海地铁日均刷卡：2 千万次
- Variety（多样性）
 - 数据形式多样，半结构化的关系数据、位置、非结构化的文本、图片、音/视频数据
 - 信息来源多样，网络数据、企事业单位数据、政府数据、媒体数据等
- Value（高价值性）
 - 大数据价值总量大
 - 知识密度低，需要通过数据分析有效地发现其价值

大数据规模



- 全球数据产生量迅猛增长
 - 2018-2020年期间，大数据市场整体收入规模保持每年约70亿美元的增长，全球大数据市场收入规模2020年达到560亿美元

- 大数据、移动互联网、物联网等产业深入发展
- 我国数据产生量
 - 从2018年的7.6ZB将增至2025年的48.6ZB



- 人类采集记录数据已有数千年历史
 - 人工采集，纸张记录，数据规模较低
 - 信息通信技术的发展，全球数据产生量迅猛增长
- 主要来源
 - 20世纪70年代，数据库管理系统广泛使用，生产、生活数据被记录存储
 - 互联网、Web2.0，自媒体催生大量原创内容
 - 物联网，传感器感知环境，产生海量数据

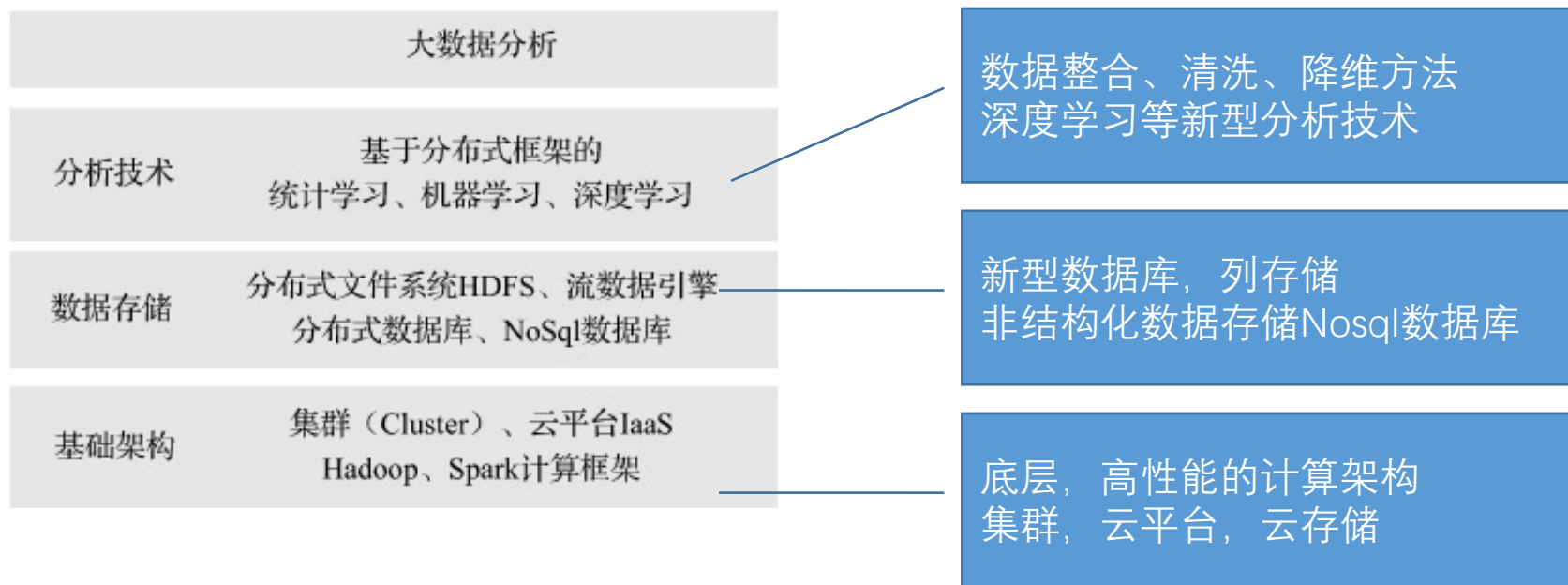
大数据影响和安全



- 技术层面，推动了分布式并行技术的创新与大规模应用
 - 催生出云平台、深度计算服务器、数据存储处理服务器、内存计算等市场
 - 引发数据快速处理分析技术、数据挖掘技术和软件产品的发展
- 社会层面，大数据处理分析成为新一代信息技术融合应用的结合点，“数据驱动”成为方向
- 信息数字化存储，带来大量安全隐患，网络空间安全，成为新战场
 - 保障安全，国家立法、严格管控
 - 全民安全教育

10.1.2 大数据技术

- 大数据属于数据科学的范畴
 - 大数据分析遵循数据科学的工作流程，继承数据分析的技术和方法
 - 当数据量达到特定规模时，需要引入分布式、并行计算、云平台等其他技术实现大规模数据的存储、计算和传输



大数据基础设施

分布式系统：多台计算机互联为一个系统，通过系统软件提供统一服务

- 超级计算机



(a) 神威 太湖之光 (10649600个处理器核)

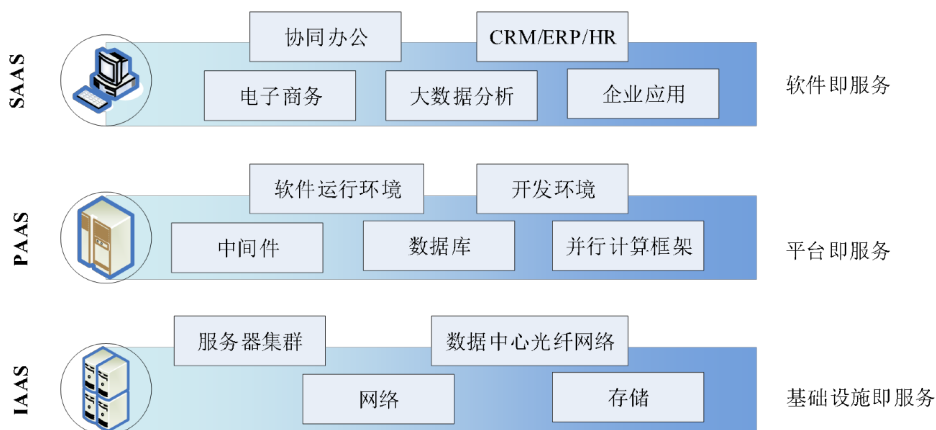


(b) 天河二号 (4981760个处理器核)

- 服务器集群

- 高性能计算集群
- 负载均衡集群
- 高可用性集群

- 云计算平台



10.2 分布式计算框架

- 分布式硬件平台上，为用户开发大数据应用提供分布式文件存储、分布式计算的编程框架
- 开源框架
 - Hadoop：分布式文件系统HDFS和分布式计算MapReduce
 - Spark：基于内存计算的并行计算框架

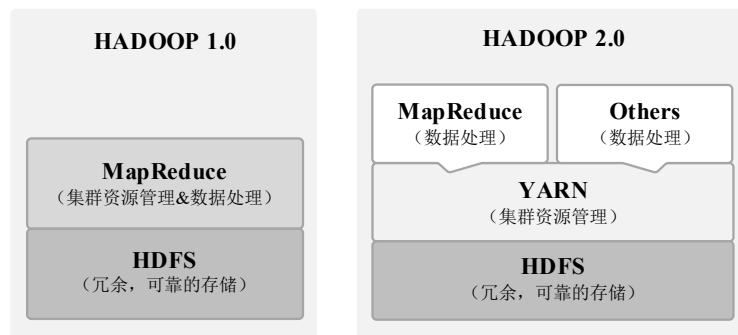


- 阿里拥有自主研发的飞天大数据计算引擎
 - 支持 10万台服务器集群
 - 离线计算、实时计算、机器学习、搜索、图计算等引擎

Hadoop概述



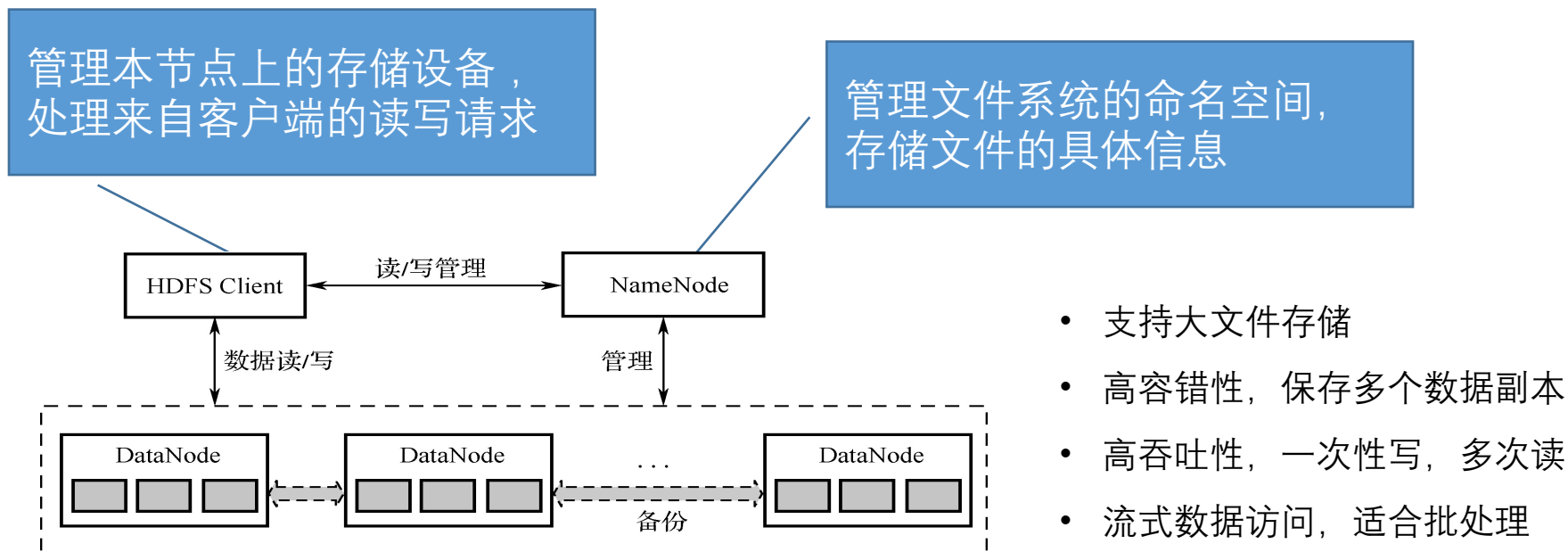
- 起源于Apache Lucence项目下的搜索引擎子项目Nutch
 - 版本 1.x, 2.x



- 采用主/从结构
 - 主节点：运行HDFS、YARN、MapReduce，响应客户端访问
 - 从节点：负责数据存储和计算
- 大规模数据处理
 - 将数据分为多份
 - 每份在从节点上存储和计算
 - 合并得到最终结果

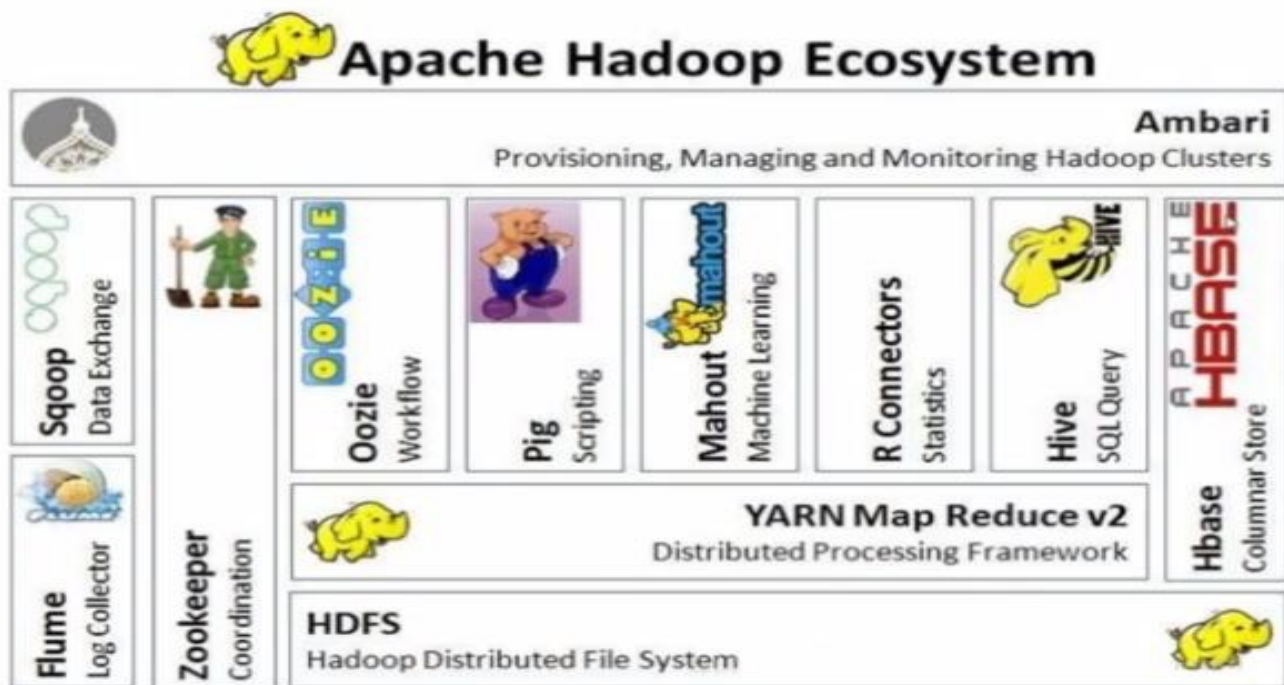
HDFS文件系统

- HDFS管理一个单独的名称节点（NameNode，运行在主节点上）和多个数据节点（DataNode，运行在从结点上）
- 将客户端（Client）上的文件数据分割成若干个数据块（Block），分布到多个数据节点上存储



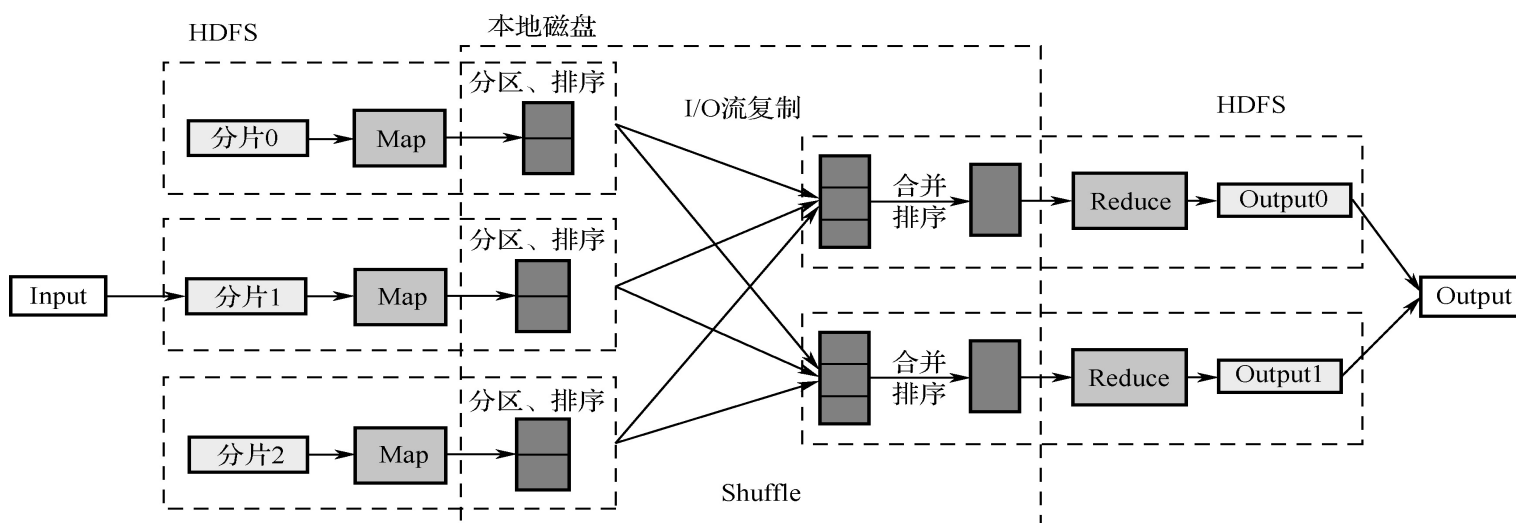
Hadoop生态圈

- 在YARN基础上逐渐发展出多种计算框架，满足不同类型大规模数据计算的需要
 - 流式计算框架Storm，内存计算框架Spark
 - 分布式数据库Hbase、数据仓库Hive
 - 可扩展机器学习框架Mahout



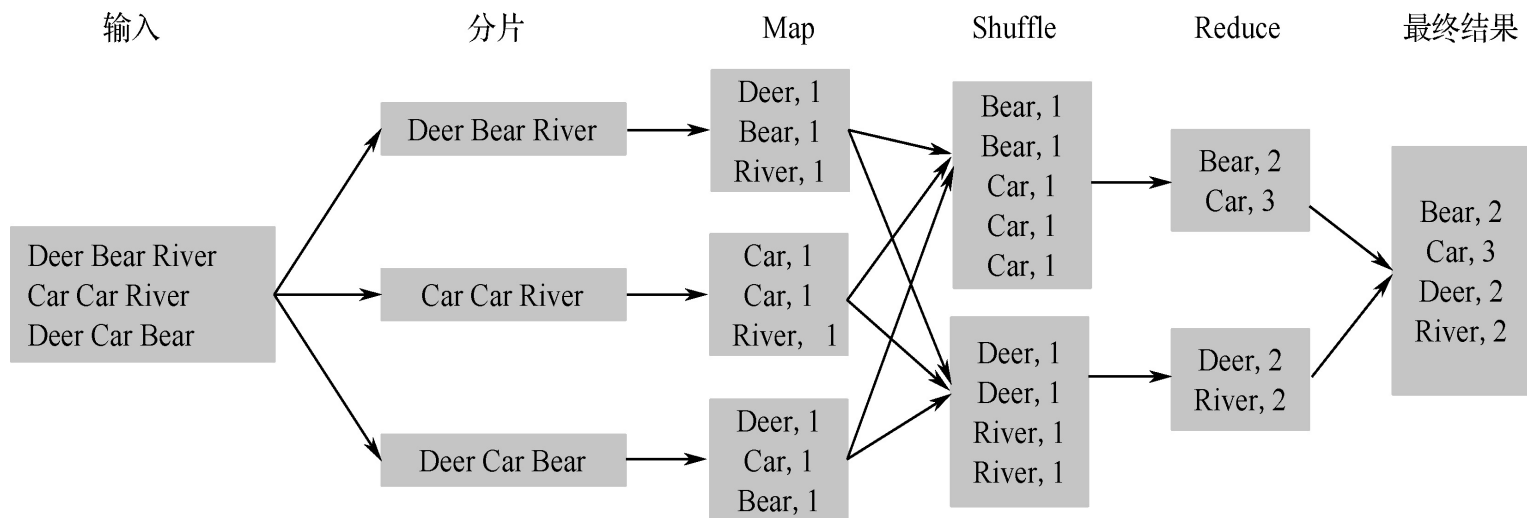
10.2.2 MapReduce

- 负责有效管理和调度集群中的节点，完成并行化程序执行和数据处理
 - 作业追踪器 (JobTracker) 负责管理和调度，部署在主节点上 (HDFS NameNode)
 - 任务追踪器 (TaskTracker) 负责数据计算，部署在从节点上 (HDFS DataNode)
- MapReduce将输入数据分片，交给不同的Map任务执行，再由Reduce任务合并得到最终结果
 - 处理过程分解为Input、Map、Shuffle、Reduce、Output等阶段
 - 用户只需定义Map和Reduce的实现方法，其余由框架完成



MapReduce计算实例

- 单词计数，统计文本中各单词出现次数
 - 逐个检查文本中出现的单词，若单词不存在，放入单词计数表，计数为1
 - 如果单词已在词表中存在，则计数加1
- 分片并行化
 - 分成多个文本片段，文本片段作为一个数据分片
 - 编程定义Map方法实现单个片段的单词解析得到词表
 - 定义Reduce方法实现多个词表的计数合并



单词计数Python实现

- Hadoop框架采用Java实现，支持Python编程
- Map 方法：输入数据来自为文件，数据处理为<key,value>形式的结果，以<Text,Int>的形式输出到文件中

```
import sys                                #引入Python系统库，对文件进行操作
for line in sys.stdin:                    #按行取出输入文件的文本
    line = line.strip()                   #去掉字符串首尾的空格
    words = line.split()                  #将每行文本分割为单词
    for word in words:
        print( "%s\t%s" % (word,1))      #输出<Text,Int>的键值对到文件
```

- Reduce 方法：输入数据为存放了多行<key,value>的文件，Reduce方法按照key对value进行求和，将得到的<key,value>集合输出到文件中。

```
import sys
word_dict = {}                            #初始化词语字典
for line in sys.stdin:                    #处理每个map的输出结果
    line = line.strip()
    word, count = line.split('\t')        #将每行文件中的key, value取出
    //若新出现的key添加到词典中，否则已有key的value加1
    word_dict[word] = word_dict.get(word, 0) +1

for k, v in word_dict.items():
    print("%s\t%s" % (k, v))
```



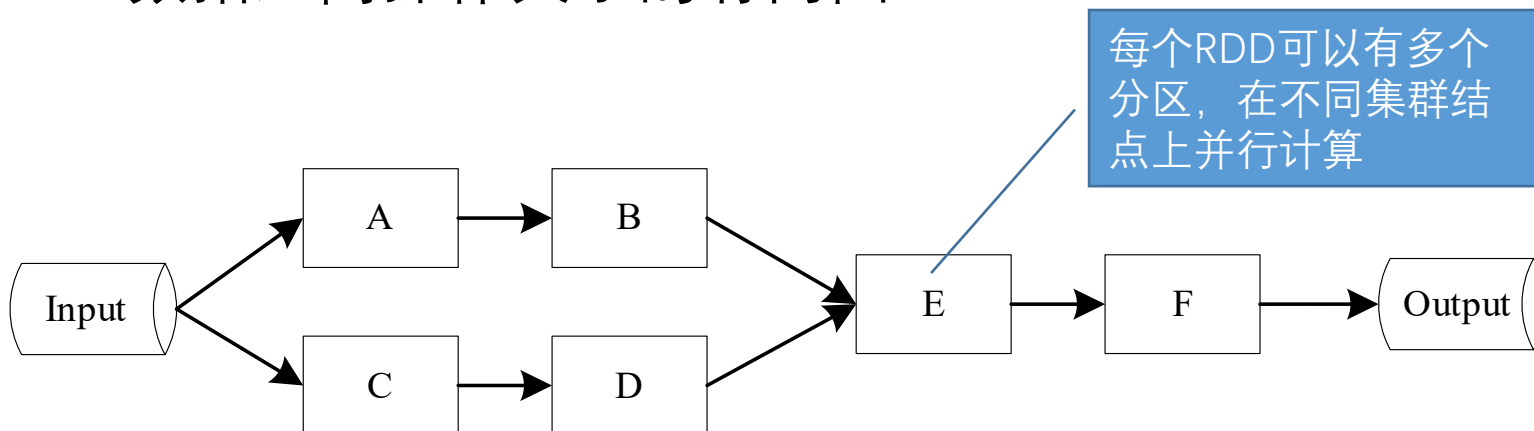
10.2.3 Spark

- Hadoop适合对大规模静态数据集合进行批量操作，缺点：
 - 计算方法表达能力有限
 - I/O文件读写开销大，延迟高
 - 难以胜任复杂、多阶段的计算任务
- Spark继承了MapReduce分布式并行的优点
 - 将中间结果放到内存中，省去大量的磁盘操作，提高处理效率
 - 更好地实现迭代式的数据分析任务，如机器学习方法
- Spark的生态系统是以Spark Core 为核心发展出的系列功能组件



Spark RDD

- Spark RDD
 - Resilient Distributed Dataset, 弹性分布数据集
 - RDD元素集合, 只能读不能直接修改
 - 提供一组丰富的操作以支持常见的数据运算
 - 大数据计算任务从逻辑上被定义为多个RDD以及数据之间操作关系的有向图



Spark DataFrame

- RDD是分布式的对象集合
 - 不定义数据对象的结构
- DataFrame数据对象，类似于pandas
 - 详细定义了每行数据的列结构
 - 支持多种数据源
 - 结构化数据文件、Hive的表、外部数据库或现有的RDDs
 - DataFrame数据可以作为关系型数据表使用
 - 通过spark.sql() 执行数据库查询语句（SQL），将查询结果作为新的DataFrame返回

Spark DataFrame实例 (1)

【例10-2】 统计哈利波特与魔法石文本中出现的单词及其频次

- Spark编程Scala, 支持Java、Python和R
- 实例采用Python语言实现

1) 读取文本文件内容创建DataFrame

```
from pyspark.sql import SparkSession
#创建应用程序WordCount
spark = SparkSession.builder.appName("WordCount").getOrCreate()

#读取txt格式的文本文件, 创建DataFrame
df = spark.read.text("data/HarryAndStone.txt")
df.show(20)  #显示DataFrame
```

Spark DataFrame实例 (2)

2) 转换字符串中所有大写字符为小写，除去标点符号，把字符串切分为单词；

```
#导入DataFrame的相关方法
from pyspark.sql.functions import lower, explode, split
#全部化为小写字母，alias()返回指定的列名，也可用f.col('value')表示列
lower_df = df.select(lower(df.value).alias('value'))

#切割字符串，根据split()返回的单词列表，使用explode()函数拆分成多行
words = lower_df.select(explode(
    split(lower_df.value, "\\W+")).alias("word"))
```

3) 统计各单词出现的次数

```
num = words.groupBy(words.word).count()
num.show()
```

10.3 分布式数据建模分布工具

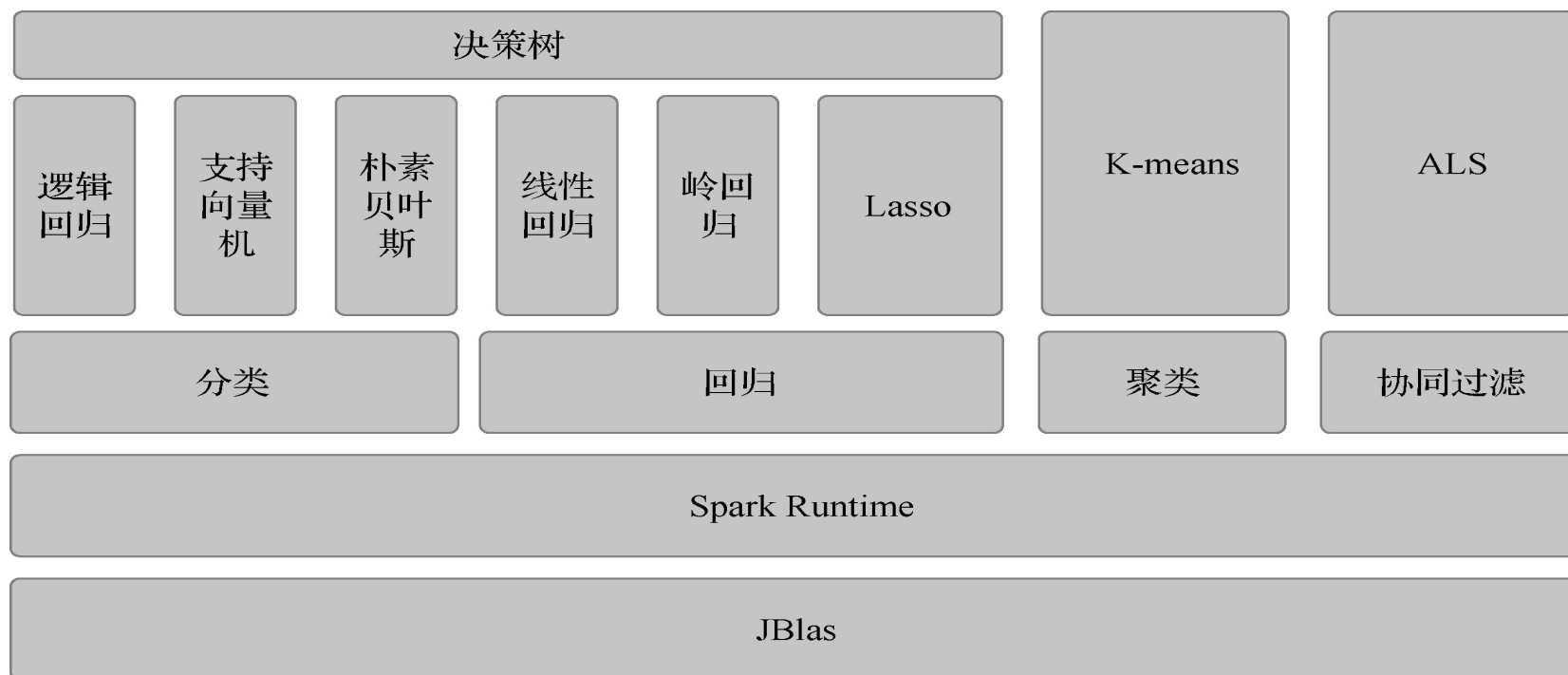
- Hadoop Mahout
 - 基于Hadoop的机器学习和数据挖掘分布式框架
 - 提供一些可扩展的机器学习领域经典算法的实现
- Spark MLlib
 - Spark的机器学习（ML）库，提供了一系列通用的机器学习算法和工具

Mahout

类别	算法名	运行模式		
		单机	Hadoop	Spark
协同过滤算法	基于用户的协同过滤算法	√		√
	基于项目的协同过滤算法	√	√	√
	基于ALS（交替最小二乘）的矩阵分解算法	√	√	
	基于隐式反馈ALS的矩阵分解算法	√	√	
	SVD++（奇异值分解）	√		
	加权矩阵分解	√		
分类算法	逻辑回归（Logistic Regression）	√		
	朴素贝叶斯/补充朴素贝叶斯		√	√
	随机森林		√	
	隐马尔科夫模型	√		
	多层感知器	√		
聚类算法	k-Means 聚类	√	√	
	模糊（Fuzzy）K-means	√	√	
	流（Streaming）K-means	√	√	
	谱（Spectral）聚类		√	
降维模型	奇异值分解（SVD）	√	√	√
	随机SVD	√	√	√
	主成分分解（PCA）	√	√	√
	正三角（QR）分解	√	√	√
主题模型	隐狄利克雷分布（LDA）	√	√	

Spark MLlib

- 由数据类型、数学统计计算库、算法评测、机器学习算法库



MLlib建模分析

- spark.ml库：基于DataFrame数据结构的API
- 单机版的机器学习算法可便捷地改写为基于MLlib的分布式系统

【例10-3】 从豆瓣网站获取了2015年中国内地上映电影的评分和评价人数，进行聚类分析。

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("movie_kmeans").getOrCreate()
#定义读取文件的数据格式
from pyspark.sql.types import *
csvSchema = StructType([StructField("rating", FloatType(), True),
                          StructField("number", FloatType(), True),])
#读取csv格式的文件，指定路径和Schema
df = spark.read.csv("data/movieRating.csv", csvSchema)
#df.show(10)

#创建转换器，使用一系列整合特征（评分和评价人数列）
from pyspark.ml.feature import VectorAssembler
featuresCreator = VectorAssembler(inputCols = ["score", "number"],
outputCol = "features")
```

建模实例 (2)

【例10-3】 从豆瓣网站获取了2015年中国内地上映电影的评分和评价人数，进行聚类分析。

```
#创建K-Means聚类模型，聚为3类
from pyspark.ml.clustering import KMeans
kmeans = KMeans(k = 3)
#训练模型
from pyspark.ml import Pipeline
pipeline = Pipeline(stages = [featuresCreator, kmeans]) #初始化管道
model = pipeline.fit(data_df) #拟合模型
predictions = model.transform(data_df) #DataFrame转换
predictions.show() #显示聚类结果

#评估模型，计算平均轮廓系数
from pyspark.ml.evaluation import ClusteringEvaluator
evaluator = ClusteringEvaluator() #初始化评估器
silhouette = evaluator.evaluate(predictions)
print(silhouette)
```


课后作业

- 查找资料，了解自己专业领域或感兴趣领域的大数据应用案例，包括应用目标、数据来源、采用的技术平台、分析技术以及应用价值等，撰写一份调研报告。